

Final Project

DPO

DPO (Direct Preference Optimization) là một phương pháp fine-tuning các mô hình ngôn ngữ lớn (LLMs) như GPT-3 hay Claude, sử dụng human preferences để align chúng với human values.

Ví dụ đơn giản: Khi bạn hỏi model "Thủ đô của Pháp là gì?" và nó đưa ra hai câu trả lời: "Paris" và "London". Nếu bạn thích "Paris" hơn, DPO giúp model học để chọn "Paris" thường xuyên hơn mà không cần các bước phức tạp như trong phương pháp truyền thống.

RLHF truyền thống có nhiều bước: model tạo câu trả lời, con người xếp hạng chúng, sau đó một "reward model" riêng được huấn luyện để chấm điểm câu trả lời, và cuối cùng model được fine-tuned bằng reinforcement learning. DPO bỏ qua reward model và trực tiếp sử dụng preferences để cập nhật model, làm cho nó nhanh hơn và đơn giản hơn.

PROJECT

DPO tối ưu hàm mất mát:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \cdot \left[\log \pi(y^+ | x) - \log \pi(y^- | x) \right] \right)$$

trong đó:

- x : lời nhắc (prompt)
- y^+ : phản hồi được ưa thích (chosen)
- y^- : phản hồi kém hơn (rejected)
- π : mô hình đang fine-tune
- σ : hàm sigmoid
- β : hệ số điều chỉnh độ nhạy

Phương pháp	Đặc điểm chính	Ưu điểm	Nhược điểm
SFT (Supervised Fine-Tuning)	Huấn luyện từ phản hồi “đúng”	Đơn giản, dễ triển khai	Không học được từ phản hồi “kém”
PPO (Proximal Policy Optimization)	RLHF với reward model	Học được từ preference	Phức tạp, khó huấn luyện
DPO	Học từ cặp (chosen, rejected)	Không cần reward model, dễ huấn luyện hơn PPO	Cần dữ liệu có cặp phản hồi rõ ràng

PROJECT

Mục tiêu chính:

Phát triển và cải thiện chất lượng phản hồi của mô hình chatbot bằng cách fine-tuning với phương pháp Direct Preference Optimization (DPO), nhằm học trực tiếp từ ưu tiên của người dùng giữa các phản hồi tốt và kém.



PROJECT

```
!pip install jsonlines
```

```
!pip install trl
```

```
!pip install -U transformers
```

```
pip install peft
```

PROJECT

```
[12]: import jsonlines
import pandas as pd

data = []
with jsonlines.open("/kaggle/input/anthropic/train.jsonl") as reader:
    for obj in reader:
        data.append(obj)

df = pd.DataFrame(data)
df.head()
```

```
[12]:
```

	chosen	rejected
0	\n\nHuman: If I was arrested for a crime, what...	\n\nHuman: If I was arrested for a crime, what...
1	\n\nHuman: What tools do I need to work on a c...	\n\nHuman: What tools do I need to work on a c...
2	\n\nHuman: What are some good exercises I can ...	\n\nHuman: What are some good exercises I can ...
3	\n\nHuman: I'm moving to Utah next month and I...	\n\nHuman: I'm moving to Utah next month and I...
4	\n\nHuman: What is an easy to make cake frosti...	\n\nHuman: What is an easy to make cake frosti...

PROJECT

```
def extract_prompt(text):
    if "Assistant:" in text:
        return text.split("Assistant:")[0].replace("Human:", "").strip()
    return ""

def remove_prompt(text):
    if "Assistant:" in text:
        return text.split("Assistant:")[1].strip()
    return text.strip()

df["prompt"] = df["chosen"].apply(extract_prompt)
df["chosen"] = df["chosen"].apply(remove_prompt)
df["rejected"] = df["rejected"].apply(remove_prompt)
```

[+ Code](#)[+ Markdown](#)

```
df[["prompt", "chosen", "rejected"]].to_json("dpo_ready_train.jsonl", orient="records", lines=True)
```

PROJECT

```
[15]: data = []
with jsonlines.open("/kaggle/working/dpo_ready_train.jsonl") as reader:
    for obj in reader:
        data.append(obj)

df = pd.DataFrame(data)
df.head()
```

[15_		prompt	chosen	rejected
0		If I was arrested for a crime, what rights do ...	Human: I am not a lawyer, so I don't have much...	Human: I am not a lawyer, so I don't have much...
1		What tools do I need to work on a car?	Generally you'll need tools like wrenches and ...	Generally you'll need tools like wrenches and ...
2		What are some good exercises I can do at a des...	You could try some of these out:\n\n\n-Avoid s...	Great question! Here are a few tips. It's al...
3		I'm moving to Utah next month and I'm curious ...	I think there are a lot of fantastic climbing ...	I think there are a lot of fantastic climbing ...
4		What is an easy to make cake frosting?	Frosting is a topping for cakes, usually made ...	Frosting is a topping for cakes, usually made ...

PROJECT

```
# Kiểm tra xem có NaN hay không
print(df.isnull().sum())

# Kiểm tra số dòng mà chosen == rejected
print((df['chosen'] == df['rejected']).sum())
```

```
prompt      0
chosen      0
rejected     0
dtype: int64
35687
```

```
# Lọc bỏ các dòng có chosen == rejected
df_cleaned = df[df['chosen'] != df['rejected']].reset_index(drop=True)
df_cleaned.to_json("/kaggle/working/dpo_ready_filtered.jsonl", orient="records", lines=True)

print(f"Số dòng ban đầu: {len(df)}")
print(f"Số dòng sau khi lọc: {len(df_cleaned)}")
```

```
Số dòng ban đầu: 52421
Số dòng sau khi lọc: 16734
```

PROJECT

```
# Kiểm tra xem có NaN hay không
print(df.isnull().sum())

# Kiểm tra số dòng mà chosen == rejected
print((df['chosen'] == df['rejected']).sum())
```

```
prompt      0
chosen      0
rejected     0
dtype: int64
35687
```

```
# Lọc bỏ các dòng có chosen == rejected
df_cleaned = df[df['chosen'] != df['rejected']].reset_index(drop=True)
df_cleaned.to_json("/kaggle/working/dpo_ready_filtered.jsonl", orient="records", lines=True)

print(f"Số dòng ban đầu: {len(df)}")
print(f"Số dòng sau khi lọc: {len(df_cleaned)}")
```

```
Số dòng ban đầu: 52421
Số dòng sau khi lọc: 16734
```

PROJECT

```
# === Load model nhỏ nhẹ để token hóa ===
model_name = "EleutherAI/pythia-70m" # hoặc: "tiiuae/falcon-rw-1b"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Đảm bảo tokenizer có pad_token
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# === Load dữ liệu gốc (file JSONL train) ===
df = pd.read_json("/kaggle/working/dpo_ready_filtered.jsonl", lines=True)
hf_dataset = Dataset.from_pandas(df)

# === Hàm tokenize chuẩn cho DPO ===
def dpo_tokenize(example):
    prompt = example["prompt"]
    chosen = example["chosen"]
    rejected = example["rejected"]

    # Tokenize prompt + chosen
    chosen_enc = tokenizer(
        prompt,
        chosen,
        truncation=True,
        padding="max_length",
        max_length=512,
    )

    # Tokenize prompt + rejected
    rejected_enc = tokenizer(
        prompt,
        rejected,
        truncation=True,
        padding="max_length",
        max_length=512,
    )

    return {
        "prompt_chosen_input_ids": chosen_enc["input_ids"],
        "prompt_chosen_attention_mask": chosen_enc["attention_mask"],
        "prompt_rejected_input_ids": rejected_enc["input_ids"],
        "prompt_rejected_attention_mask": rejected_enc["attention_mask"],
```

```
# === Áp dụng tokenize toàn bộ dataset ===
tokenized_dataset = hf_dataset.map(dpo_tokenize, batched=False)

# === Lưu ra file nếu cần (ở Kaggle working) ===
tokenized_dataset.to_json("/kaggle/working/tokenized_dpo.json")
```

PROJECT

```
# === Load dữ liệu đã tokenize ===
df_tok = pd.read_json("/kaggle/working/tokenized_dpo.json", lines=True)
tokenized_dataset = Dataset.from_pandas(df_tok)
dataset = tokenized_dataset.train_test_split(test_size=0.05, seed=42)
train_dataset = dataset["train"]
eval_dataset = dataset["test"]

# === Load tokenizer & base model ===
model_name = "EleutherAI/pythia-70m"
tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# === Load model gốc cho ref_model (KHÔNG áp dụng LoRA) ===
ref_model = AutoModelForCausalLM.from_pretrained(model_name)

# === Load lại model chính & áp dụng LoRA ===
base_model = AutoModelForCausalLM.from_pretrained(model_name)
lora_config = LoraConfig(
    r=8,
    lora_alpha=32,
    task_type=TaskType.CAUSAL_LM,
    lora_dropout=0.05,
    bias="none"
)
model = get_peft_model(base_model, lora_config)

# === Callback ghi logs ===
class LossLoggerCallback(TrainerCallback):
    def __init__(self):
        self.logs = []

    def on_log(self, args, state, control, logs=None, **kwargs):
        if logs:
            logs['step'] = state.global_step
            self.logs.append(logs)

loss_logger = LossLoggerCallback()
```

```
# === Cấu hình huấn luyện ===
training_args = DPOConfig(
    output_dir="/kaggle/working/dpo-output",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    logging_steps=50,
    eval_strategy='steps',
    eval_steps=100,
    save_strategy="no",
    report_to="none",
    beta=0.3,
    max_prompt_length=256,
    max_length=200,
    padding_value=tokenizer.pad_token_id,
)

# === Khởi tạo trainer ===
trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    callbacks=[loss_logger],
)

# === Huấn luyện ===
trainer.train()

# === Lưu model fine-tuned ===
save_path = "/kaggle/working/dpo-finetuned-model"
trainer.model.save_pretrained(save_path)
tokenizer.save_pretrained(save_path)
```

PROJECT

Trong DPO, ta huấn luyện mô hình để **thích phản hồi tốt (chosen)** và **không thích phản hồi kém (rejected)**.

Công thức loss:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \cdot [\log \pi(y^+ | x) - \log \pi(y^- | x)] \right)$$

Bạn tưởng tượng:

Mô hình sẽ học **tăng** xác suất của phản hồi tốt y^+ , và **giảm** xác suất của phản hồi xấu y^- với cùng một prompt x .

Thông thường, mỗi lớp trong mô hình có trọng số (weights) như:

$$\text{Output} = W \cdot x$$

Khi fine-tune toàn bộ mô hình (DPO không dùng LoRA), thì **toàn bộ W** sẽ được cập nhật → rất nặng và tốn RAM.

LoRA thay thế bằng:

$$\text{Output} = W \cdot x + (B \cdot A) \cdot x$$

- W : giữ nguyên, không cập nhật
- A, B : là hai ma trận nhỏ, bạn chỉ cập nhật chúng

💡 Tức là: **thay vì học lại toàn bộ trọng số**, bạn **chỉ học một phần rất nhỏ**, nhưng đủ để điều chỉnh mô hình.

PROJECT

Original (ref_model) output:

What are some good habits to stay productive during the day?

I've always enjoyed the fact that I can enjoy the same experience for over a year, and I've learned the things in life that are not "being productive" and that are "always a constant, always an enjoyable and enjoyable experience".

I didn't get it off, so I decided to focus on the fact that my family and I have a great job with the work, and while that is a difficult task, I am still not happy with the way that

DPO-finetuned model output:

What are some good habits to stay productive during the day?

A:

The main problem with regular maintenance of your daily routine is that it takes time to get to know how to maintain your daily routines.

You need to spend more time on the task. You need to think about how you plan to maintain your daily routine.

A:

You need to think about how you plan to maintain your daily routine during your day.

